UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS



Facultad de Ingeniería
Carrera de Ciencias de la Computación

TEORÍA DE COMPILADORES (1ACC0218)

Sección: 4889
Docente: Jose Luis Soncco Alvarez

Kipu

Un lenguaje de dominio específico para reglas tributarias y montos monetarios

Hito 2: Gramática actualizada, analizador semántico completo, arquitectura, plan de validación y avance de generación de código con LLVM

Integrantes

Bussalleu Salcedo, Fabrizio (U202315655)
Sancho Navarro, Jose Luis (U202016523)
Cabrera Díaz, Marco Antonio Luciano (U202318540)

https://github.com/FabrizioBussalleu/kipu
Lima, noviembre de 2026

Índice

1. Problemática y motivación **2**
2. Objetivos **2**
3. Marco conceptual **3**
4. Construcciones del lenguaje **4**
5. Analizador léxico **6**
6. Analizador sintáctico **7**
6.1. Gramática actualizada 7
6.2. Validación por derivaciones más a la izquierda 9
7. Analizador semántico **9**
8. Arquitectura del compilador **11**
9. Plan de validación del compilador **12**
10. Avance de la implementación: generación de LLVM IR **14**
11. Implementación, repositorio y uso **15**
12. Próximos pasos (Hito 3) **16**
Referencias **16**

1. Problemática y motivación

En el Perú, gran parte de las operaciones comerciales formales está sujeta a reglas tributarias como el IGV, las detracciones del SPOT, las retenciones y las percepciones; además, muchas operaciones requieren trabajar con distintos tipos de cambio. En pequeñas empresas y en sistemas de facturación electrónica, estas reglas suelen manejarse en hojas de cálculo o implementarse en lenguajes de propósito general. Las condiciones y porcentajes del sistema de detracciones dependen del tipo de operación y de los criterios definidos por SUNAT (SUNAT, s. f.).

Este enfoque genera varios problemas. El primero es el manejo de decimales y redondeos: la aritmética de punto flotante no representa con exactitud los céntimos (por ejemplo, 0.1 + 0.2 no es exactamente 0.3). El segundo es que un lenguaje de propósito general no impide, por sí solo, mezclar montos en soles y dólares. El tercero es que las reglas de negocio quedan repartidas dentro del código, lo que dificulta su lectura y auditoría por parte de personas que no programan. Finalmente, muchos de estos errores se detectan recién cuando el programa se ejecuta, o incluso durante una fiscalización.

Las herramientas existentes resuelven partes del problema: las hojas de cálculo no tipan monedas, los motores de reglas genéricos no conocen el concepto de monto y las bibliotecas de dinero validan la moneda en tiempo de ejecución. Por ese motivo se propone **Kipu**, un lenguaje de dominio específico inspirado en el quipu, el instrumento contable inca. Kipu incorpora tipos como `monto<PEN>`, `monto<USD>` y `porcentaje`, de modo que el compilador detecta antes de la ejecución las operaciones sin sentido contable y genera código que opera con céntimos exactos.

2. Objetivos

Objetivo general. Diseñar e implementar un compilador para Kipu que traduzca reglas tributarias escritas en español a código ejecutable mediante ANTLR4 y LLVM, verificando durante la compilación la coherencia de tipos y monedas.

Objetivos específicos. La Tabla 1 resume los objetivos específicos y su estado al cierre del Hito 2.

Tabla 1. Objetivos específicos y estado de avance

Código	Objetivo específico	Estado (Hito 2)
OE1	Definir el léxico y las construcciones de Kipu, con tipos de dominio (`monto<moneda>`, `porcentaje`) además de los tipos básicos.	Completo
OE2	Especificar la gramática libre de contexto en la forma estándar del curso y validarla con derivaciones más a la izquierda.	Completo (gramática actualizada)
OE3	Implementar los analizadores léxico y sintáctico en ANTLR4, con mensajes de error en español, línea y columna.	Completo
OE4	Implementar el analizador semántico completo con tabla de símbolos por ámbitos y sistema de tipos monetario.	Completo (14 errores)
OE5	Generar LLVM IR representando los montos como enteros de 64 bits en céntimos.	En avance (Sección 10); se completa en el Hito 3

3. Marco conceptual

Esta sección define los conceptos del curso que se aplican en el compilador de Kipu y señala dónde se usa cada uno. Las definiciones siguen a Aho et al. (2007) y, para la herramienta, a Parr (2013).

Tabla 2. Conceptos del curso aplicados en Kipu

Concepto	Definición	Aplicación en Kipu
Compilador	Programa que lee un programa en un lenguaje fuente y lo traduce a un programa equivalente en otro lenguaje (objeto o intermedio), informando los errores que encuentre.	Traduce programas `.kp` a LLVM IR.
Front end y back end	El front end (análisis) divide el programa en sus partes y verifica su corrección; el back end (síntesis) construye el programa destino a partir de la representación intermedia.	Front end: léxico, sintáctico, AST y semántico. Back end: generador LLVM IR.
Token	Par formado por un nombre de token (categoría abstracta) y un valor de atributo opcional; es la unidad que el analizador léxico entrega al sintáctico.	`ID`, `MONEDA`, `LIT_PORCENTAJE`, `PUNTO_COMA`.
Lexema	Secuencia de caracteres del programa fuente que coincide con el patrón de un token.	`subtotal`, `PEN`, `18%`, `;`.
Patrón	Descripción de la forma que pueden tener los lexemas de un token; se expresa con expresiones regulares.	Reglas del lexer en Kipu.g4.
Analizador léxico	Primera fase: lee los caracteres de entrada, los agrupa en lexemas y produce la secuencia de tokens; descarta espacios y comentarios.	KipuLexer, generado por ANTLR4.
Fragment (ANTLR)	Regla léxica auxiliar que no genera un token por sí misma; se usa para componer otros tokens.	`DIGITOS`, `ID_COLA`, `CADENA_COLA`.
Gramática libre de contexto	Cuádrupla G = (N, T, P, S): no terminales, terminales, producciones de la forma A → α y un símbolo inicial.	37 no terminales, 52 tokens y 92 producciones; S = `programa`.
Producción ε	Producción cuyo cuerpo es la cadena vacía; permite expresar partes opcionales sin el operador ?.	`inicializacion → ASIG expresion | ε`.
Recursión y listas	Una lista se describe con una producción recursiva y una alternativa vacía, en lugar de los operadores * y +.	`listaSentencias → sentencia listaSentencias | ε`.
Derivación más a la izquierda	Derivación en la que, en cada paso, se reemplaza el no terminal situado más a la izquierda de la forma sentencial.	28 derivaciones (Anexo A).
Árbol de análisis sintáctico	Representación gráfica de una derivación: la raíz es el símbolo inicial, los nodos internos son no terminales y las hojas son terminales.	Lo construye KipuParser; opción `--arbol`.
Ambigüedad	Una gramática es ambigua si alguna cadena tiene más de un árbol de análisis.	Se verifica con la detección exacta de ambigüedad de ANTLR.

Concepto	Definición	Aplicación en Kipu
Precedencia y asociatividad	Se codifican con un no terminal por nivel de precedencia; la recursión por la izquierda produce asociatividad izquierda.	`expresion`, `conjuncion`, ..., `termino`, `unario`.
Analizador sintáctico	Fase que verifica que la secuencia de tokens pueda generarse con la gramática y construye el árbol de análisis.	KipuParser, parser ALL(*) de ANTLR4.
Árbol de sintaxis abstracta (AST)	Representación del programa que conserva solo la estructura significativa, sin los símbolos auxiliares de la gramática.	Módulos `ast_kipu.py` y `constructor_ast.py`.
Análisis semántico	Fase que verifica las restricciones que la gramática libre de contexto no puede expresar, como declarar antes de usar o la compatibilidad de tipos.	14 errores semánticos (Sección 7).
Tabla de símbolos	Estructura que guarda, por cada identificador, su categoría, tipo, ubicación y demás atributos.	Clase `TablaSimbolos`; opción `--simbolos`.
Ámbito	Región del programa en la que una declaración es visible; los ámbitos anidados permiten ocultar nombres externos.	Global, regla, bloque y para.
Sistema de tipos y coerción	Conjunto de reglas que asigna un tipo a cada expresión; una coerción (ensanchamiento) convierte un tipo en otro de forma implícita y segura.	Reglas de la Tabla 7; `entero` se ensancha a `decimal`.
Representación intermedia y LLVM IR	Código entre el fuente y la máquina. LLVM IR es una representación tipada, en forma SSA (cada valor se asigna una sola vez) y organizada en bloques básicos.	Generador `generador_llvm.py` (llvmlite).
Bloque básico y JIT	Un bloque básico es una secuencia de instrucciones sin saltos internos que termina en un salto. Un compilador JIT traduce el IR a código máquina en tiempo de ejecución.	`si`, `mientras` y `para` generan bloques; `--ejecutar` usa MCJIT.

4. Construcciones del lenguaje

Kipu utiliza tipado estático y explícito, bloques delimitados por llaves y sentencias terminadas en punto y coma. La precedencia de operadores, de mayor a menor, es: paréntesis, llamadas y `convertir`; menos unario; `*`, `/` y `mod`; `+` y `-`; operadores relacionales; igualdad; `no`; `y`; `o`. Los montos pueden multiplicarse por números o porcentajes, y el cambio de moneda se realiza únicamente con `convertir(monto, moneda, tipo_cambio)`. La Tabla 3 presenta las siete construcciones con cinco ejemplos cada una; los 35 ejemplos se validan automáticamente con el parser.

Tabla 3. Construcciones de Kipu y ejemplos

Construcción	Ejemplos (5 por construcción)

G1. Declaración de variables y constantes
`var` o `const`, nombre, tipo obligatorio y valor inicial (opcional en variables, obligatorio en constantes).
	`var cantidad: entero = 3;`
	`const IGV: porcentaje = 18%;`
	`var subtotal: monto<PEN> = 1250.50 PEN;`
	`var razonSocial: texto = "Elaris SAC";`
	`var esAgenteRetencion: booleano;`

G2. Asignación
Asignación simple (`=`) y compuesta (`+=`, `-=`).
	`cantidad = 5;`
	`subtotal = precio * cantidad;`
	`total += igv;`
	`saldo -= 150.00 PEN;`
	`exonerado = falso;`

G3. Expresiones aritméticas, monetarias, relacionales y lógicas
Aritméticas, monetarias, relacionales, lógicas y conversión de moneda.
	`precio * cantidad + flete`
	`base * 18%`
	`(total - detraccion) / 2`
	`total >= 700.00 PEN y no exonerado`
	`convertir(neto, USD, 3.75)`

G4. Sentencias selectivas
`si` con cero o más `sino si` y un `sino` opcional; bloques obligatorios.
	`si (total > 700.00 PEN) { detraccion = total * 12%; }`
	`si (esExportacion) { igv = 0.00 PEN; } sino { igv = base * 18%; }`
	`si (tipo == 1) { tasa = 12%; } sino si (tipo == 2) { tasa = 10%; } sino { tasa = 4%; }`
	`si (no pagado y diasAtraso > 30) { mostrar("Factura vencida"); }`
	`si (saldo <= 0.00 PEN) { cancelado = verdadero; } sino { saldo -= cuota; }`

G5. Sentencias iterativas
`mientras` con condición y `para` con contador, límites y `paso` opcional.
	`mientras (saldo > 0.00 PEN) { saldo -= cuota; }`
	`para mes desde 1 hasta 12 { total += cuota; }`
	`para i desde 1 hasta n paso 2 { mostrar(i); }`
	`mientras (intentos < 3 y no validado) { intentos += 1; }`
	`para anio desde 2020 hasta 2026 { si (anio mod 4 == 0) { dias = 366; } }`

G6. Reglas (funciones), retorno y llamadas
Funciones tipadas con parámetros, `retornar` y llamada como expresión o sentencia.
	`regla calcularIgv(base: monto<PEN>): monto<PEN> { retornar base * 18%; }`
	`regla esMayor(a: entero, b: entero): booleano { retornar a > b; }`
	`regla imprimirCabecera() { mostrar("FACTURA ELECTRONICA"); }`
	`regla cuota(total: monto<PEN>, n: entero): monto<PEN> { retornar total / n; }`
	`regla neto(total: monto<PEN>, tasa: porcentaje): monto<PEN> { retornar total - calcularDescuento(total, tasa); }`

G7. Entrada y salida
`mostrar` con uno o más argumentos y `leer` sobre una variable.
	`mostrar("Total:", total);`
	`mostrar(subtotal + igv);`
	`leer(cantidad);`
	`mostrar("IGV", calcularIgv(base), "Neto", neto);`
	`leer(precioUnitario);`

5. Analizador léxico

El léxico se define en la sección de lexer de `grammar/Kipu.g4` siguiendo los lineamientos del curso. Todo símbolo especial, operador y palabra reservada es un token con nombre, y las palabras reservadas se declaran antes que `ID` para que tengan prioridad. Cuando dos tokens comparten un prefijo, ANTLR aplica la coincidencia más larga (`<=` frente a `<`; `sinoCaso` es un `ID`). Los tokens compuestos no usan los operadores `*`, `+` ni `?`: se construyen con *fragments* recursivos, por ejemplo `ID : LETRA ID_COLA` con `ID_COLA : LETRA ID_COLA | DIGITO ID_COLA | ε`. El comentario de bloque se eliminó en este hito; solo existe el comentario de línea `//`.

Tabla 4. Léxico de Kipu (token: lista de lexemas)

VAR: var	**MOD**: mod
CONST: const	**MAS_ASIG**: +=
ENTERO: entero	**MENOS_ASIG**: -=
DECIMAL: decimal	**IGUAL**: ==
PORCENTAJE: porcentaje	**DIFERENTE**: !=
BOOLEANO: booleano	**MENOR_IGUAL**: <=
TEXTO: texto	**MAYOR_IGUAL**: >=
MONTO: monto	**MENOR**: <
MONEDA: PEN, USD, EUR	**MAYOR**: >
SI: si	**ASIG**: =
SINO: sino	**SUMA**: +
MIENTRAS: mientras	**RESTA**: -
PARA: para	**MULT**: *
DESDE: desde	**DIV**: /
HASTA: hasta	**PAR_IZQ**: (
PASO: paso	**PAR_DER**:)
REGLA: regla	**LLAVE_IZQ**: {
RETORNAR: retornar	**LLAVE_DER**: }
MOSTRAR: mostrar	**PUNTO_COMA**: ;
LEER: leer	**COMA**: ,
CONVERTIR: convertir	**DOS_PUNTOS**: :
Y: y	**LIT_PORCENTAJE**: DIGITOS % | DIGITOS . DIGITOS % (18%, 12.5%)
O: o	**NUM_DECIMAL**: DIGITOS . DIGITOS (3.75, 1250.50)
NO: no	**NUM_ENTERO**: DIGITOS (0, 3, 2026)

VERDADERO: verdadero	**CADENA**: " CADENA_COLA " ("Elaris SAC")
FALSO: falso	**ID**: LETRA ID_COLA (subtotal, IGV, año)

Fragments: `DIGITOS : DIGITO DIGITOS_COLA`; `DIGITOS_COLA : DIGITO DIGITOS_COLA | ε`; `ID_COLA : LETRA ID_COLA | DIGITO ID_COLA | ε`; `CADENA_COLA : CARACTER_CADENA CADENA_COLA | ε`; `DIGITO : [0-9]`; `LETRA : [a-zA-Z_áéíóúÁÉÍÓÚñÑ]`. Tokens ignorados: **COMENTARIO**: `// ...` hasta el fin de línea; **ESPACIO**: espacio, tabulación y salto de línea. Un símbolo fuera del alfabeto (`#`, `@`, `$`) o una cadena sin cerrar produce el error **LEX-01** con línea y columna.

6. Analizador sintáctico

6.1. Gramática actualizada

En el Hito 1, `Kipu.g4` usaba operadores EBNF (`?`, `*`, `+`), subreglas agrupadas, etiquetas `op=(...)` y literales dentro del parser. En este hito la gramática se reescribió completa en la forma estándar del curso. La Tabla 5 resume los cambios y la verificación automática `scripts/verificar_gramatica.py` comprueba los seis lineamientos en cada ejecución de pruebas.

Tabla 5. Cambios de la gramática respecto al Hito 1

Lineamiento	Hito 1	Hito 2
L1. Símbolos como tokens con nombre	Literales en el parser en algunas reglas	El parser solo usa nombres de token (`LLAVE_IZQ`, `PUNTO_COMA`, `MULT`)
L2. Sin operador ?	`(ASIG expresion)?`	`inicializacion → ASIG expresion | ε`
L3. Sin * ni + (parser)	`sentencia*`, `parametro (COMA parametro)*`	`listaSentencias`, `parametros` y `parametrosCola` recursivas con ε
L3. Sin * ni + (lexer)	`DIGITO+`, `LETRA (LETRA | DIGITO)*`	Fragments recursivos `DIGITOS`, `ID_COLA`
L4. Sin subreglas ni op=	`expresion op=(MULT|DIV|MOD) expresion` con etiquetas #	Niveles: `termino → termino MULT unario | termino DIV unario | termino MOD unario | unario`
L5. Reservadas antes que ID	Cumplía	Cumple
L6. Sin comentario de bloque	`COMENTARIO_BLOQUE : '/*' .*? '*/'`	Eliminado; solo `//`

La Tabla 6 presenta la gramática completa. Se usa la notación de producciones: no terminales en *cursiva*, terminales (nombres de token del lexer) en **negrita**, → para separar cabeza y cuerpo, | para alternativas y ε para la cadena vacía. Como `Kipu.g4` ya no usa operadores EBNF, la gramática del informe coincide regla por regla con la implementación; la tabla se generó a partir del propio archivo `.g4`.

Tabla 6. Gramática libre de contexto de Kipu (Hito 2)

G0. Estructura general del programa

programa → *listaSentencias* **EOF**

listaSentencias → *sentencia listaSentencias* | ε

sentencia → *declaracion* | *asignacion* | *seleccion* | *iteracion* | *regla* | *retorno* | *entradaSalida* | *llamadaSentencia* | *bloque*

bloque → **LLAVE_IZQ** *listaSentencias* **LLAVE_DER**

G1. Declaración de variables y constantes

declaracion → **VAR ID DOS_PUNTOS** *tipo inicializacion* **PUNTO_COMA** | **CONST ID DOS_PUNTOS** *tipo* **ASIG** *expresion* **PUNTO_COMA**

inicializacion → **ASIG** *expresion* | ε

tipo → **ENTERO** | **DECIMAL** | **PORCENTAJE** | **BOOLEANO** | **TEXTO** | **MONTO MENOR** **MONEDA MAYOR**

G2. Asignación

asignacion → **ID** *opAsignacion expresion* **PUNTO_COMA**

opAsignacion → **ASIG** | **MAS_ASIG** | **MENOS_ASIG**

G3. Expresiones

expresion → *expresion* **O** *conjuncion* | *conjuncion*

conjuncion → *conjuncion* **Y** *negacion* | *negacion*

negacion → **NO** *negacion* | *igualdad*

igualdad → *igualdad* **IGUAL** *relacional* | *igualdad* **DIFERENTE** *relacional* | *relacional*

relacional → *relacional* **MENOR** *aritmetica* | *relacional* **MENOR_IGUAL** *aritmetica* | *relacional* **MAYOR** *aritmetica* | *relacional* **MAYOR_IGUAL** *aritmetica* | *aritmetica*

aritmetica → *aritmetica* **SUMA** *termino* | *aritmetica* **RESTA** *termino* | *termino*

termino → *termino* **MULT** *unario* | *termino* **DIV** *unario* | *termino* **MOD** *unario* | *unario*

unario → **RESTA** *unario* | *primario*

primario → **PAR_IZQ** *expresion* **PAR_DER** | **CONVERTIR PAR_IZQ** *expresion* **COMA MONEDA** **COMA** *expresion* **PAR_DER** | *llamada* | *literal* | **ID**

literal → *numero* **MONEDA** | **NUM_ENTERO** | **NUM_DECIMAL** | **LIT_PORCENTAJE** | **CADENA** | **VERDADERO** | **FALSO**

numero → **NUM_ENTERO** | **NUM_DECIMAL**

llamada → **ID PAR_IZQ** *argumentos* **PAR_DER**

argumentos → *expresion argumentosCola* | ε

argumentosCola → **COMA** *expresion argumentosCola* | ε

G4. Sentencias selectivas

seleccion → **SI PAR_IZQ** *expresion* **PAR_DER** *bloque parteSino*

parteSino → **SINO** *colaSino* | ε

colaSino → *seleccion* | *bloque*

G5. Sentencias iterativas

iteracion → **MIENTRAS PAR_IZQ** *expresion* **PAR_DER** *bloque* | **PARA ID DESDE** *expresion* **HASTA** *expresion pasoOpcional bloque*

pasoOpcional → **PASO** *expresion* | ε

G6. Reglas (funciones), retorno y llamadas

regla → **REGLA ID PAR_IZQ** *parametros* **PAR_DER** *tipoRetorno bloque*

parametros → *parametro parametrosCola* | ε

parametrosCola → **COMA** *parametro parametrosCola* | ε

parametro → **ID DOS_PUNTOS** *tipo*
tipoRetorno → **DOS_PUNTOS** *tipo* | ε
retorno → **RETORNAR** *valorRetorno* **PUNTO_COMA**
valorRetorno → *expresion* | ε
llamadaSentencia → *llamada* **PUNTO_COMA**

G7. Entrada y salida

entradaSalida → **MOSTRAR PAR_IZQ** *expresion argumentosCola* **PAR_DER PUNTO_COMA** | **LEER PAR_IZQ ID PAR_DER PUNTO_COMA**

6.2. Validación por derivaciones más a la izquierda

Para cada gramática (G1 a G7) se derivaron por la izquierda cuatro de sus cinco ejemplos, con un total de 28 derivaciones. Como la gramática está en forma estándar, el árbol de análisis de ANTLR corresponde exactamente a la gramática de la Tabla 6: recorrerlo en preorden, reemplazando siempre el no terminal de más a la izquierda, produce la derivación más a la izquierda. El script `scripts/derivaciones.py` además exige que no haya errores, que se consuma toda la entrada, que cada paso use una producción existente en `Kipu.g4` y que la detección exacta de ambigüedad de ANTLR no reporte ningún caso. Las 28 derivaciones completas están en el **Anexo A**. Como ejemplo, la derivación de G2.1, `cantidad = 5;`, es:

> *asignacion* ⇒lm **ID** *opAsignacion expresion* **PUNTO_COMA** ⇒lm **ID ASIG** *expresion* **PUNTO_COMA** ⇒lm **ID ASIG** *conjuncion* **PUNTO_COMA** ⇒lm **ID ASIG** *negacion* **PUNTO_COMA** ⇒lm **ID ASIG** *igualdad* **PUNTO_COMA** ⇒lm **ID ASIG** *relacional* **PUNTO_COMA** ⇒lm **ID ASIG** *aritmetica* **PUNTO_COMA** ⇒lm **ID ASIG** *termino* **PUNTO_COMA** ⇒lm **ID ASIG** *unario* **PUNTO_COMA** ⇒lm **ID ASIG** *primario* **PUNTO_COMA** ⇒lm **ID ASIG** *literal* **PUNTO_COMA** ⇒lm **ID ASIG NUM_ENTERO PUNTO_COMA**

7. Analizador semántico

El análisis semántico recorre el AST (Sección 8) y mantiene una tabla de símbolos con ámbitos anidados: el ámbito global, el de cada regla (que comparte ámbito con sus parámetros), el de cada bloque y el de cada `para` (que comparte ámbito con su contador). Las reglas globales se registran en una primera pasada, lo que permite llamarlas antes de su definición. Cada expresión recibe un tipo según las reglas de la Tabla 7; cuando una subexpresión tiene error, su tipo se marca como desconocido para no generar errores en cascada.

Tabla 7. Reglas principales del sistema de tipos

Operación	Tipo resultante
`monto<M> + monto<M>`, `monto<M> - monto<M>`	`monto<M>` (monedas distintas: SEM-03)
`monto * entero | decimal | porcentaje` (en cualquier orden)	`monto` de la misma moneda
`monto<M> / entero | decimal`	`monto<M>`
`monto<M> / monto<M>`	`decimal`
`entero op entero` (`+ - * mod`)	`entero`; la división `/` produce `decimal`
`entero op decimal`	`decimal` (ensanchamiento de entero)
`porcentaje ± porcentaje`, `porcentaje * número`	`porcentaje`

Operación	Tipo resultante
`convertir(monto<A>, B, tipo_cambio)`	`monto<B>`; el tipo de cambio es entero o decimal
Relacionales y de igualdad	`booleano` (montos de distinta moneda: SEM-03)
`y`, `o`, `no`	`booleano`; sus operandos deben ser booleanos

La Tabla 8 lista los 14 errores semánticos verificados (el hito exige al menos seis). Cada uno tiene al menos dos casos de prueba negativos en `tests/semantico/invalidos`.

Tabla 8. Errores semánticos verificados

Código	Error	Ejemplo	Mensaje del compilador
SEM-01	Identificador no declarado (variable, constante o regla)	`total = subtotal + igv;`	'igv' no ha sido declarado.
SEM-02	Redeclaración en el mismo ámbito (incluye parámetros y contador del para)	`para mes ... { var mes: entero = 1; }`	'mes' ya fue declarado en este ámbito (línea 5).
SEM-03	Monedas incompatibles al operar, comparar o asignar	`soles + dolares`	No se puede sumar monto<PEN> con monto<USD>; use convertir(...).
SEM-04	Tipos incompatibles en asignación u operación	`var n: monto<PEN> = "Elaris";`	No se puede asignar texto a 'n' de tipo monto<PEN>.
SEM-05	Modificación de una constante (asignación o leer)	`IGV = 10%;`	No se puede modificar la constante 'IGV'.
SEM-06	Condición no booleana en si, sino si o mientras	`si (cantidad) { ... }`	La condición de 'si' debe ser booleana; se recibió entero.
SEM-07	retornar fuera de una regla	`retornar total;` (a nivel global)	'retornar' solo puede usarse dentro de una regla.
SEM-08	Valor de retorno incompatible con el tipo declarado	`retornar convertir(base, USD, 0.27);` en regla monto<PEN>	La regla 'igvSoles' retorna monto<PEN> y se intenta retornar monto<USD>.
SEM-09	Número de argumentos incorrecto	`calcularIgv()`	La regla 'calcularIgv' espera 1 argumento(s) y recibió 0.
SEM-10	Tipo de argumento incompatible	`cuota(1200.00 PEN, 2.5)`	Argumento 2 de 'cuota': se esperaba entero y se recibió decimal.
SEM-11	Regla con tipo de retorno que no retorna en todos los caminos	`si` sin `sino` que retorna solo en una rama	La regla 'detraccion' no retorna un valor en todos los caminos.
SEM-12	Porcentaje literal fuera del rango 0 % a 100 %	`const RECARGO: porcentaje = 150%;`	Porcentaje fuera de rango: 150%.
SEM-13	División o módulo entre cero literal (incluye tipo de cambio cero)	`total / 0`	División entre cero: el divisor de '/' es el literal 0.

Código	Error	Ejemplo	Mensaje del compilador
SEM-14	Regla sin valor de retorno usada en una expresión	`var x: entero = cabecera();`	La regla 'cabecera' no retorna un valor y no puede usarse en una expresión.

Para decidir SEM-11 se aplica un análisis de caminos sobre el AST: una lista de sentencias retorna siempre si contiene un `retornar`, un bloque que retorna siempre, o un `si` con `sino` en el que todas las ramas retornan siempre. Los ciclos no garantizan el retorno, porque su cuerpo puede no ejecutarse.

8. Arquitectura del compilador

El compilador se organiza como una cadena de fases. Cada fase recibe la representación producida por la anterior y reporta sus errores al gestor común con código, línea y columna. Si una fase encuentra errores, las siguientes no se ejecutan. La Figura 1 muestra la arquitectura y la Tabla 9 describe los módulos.

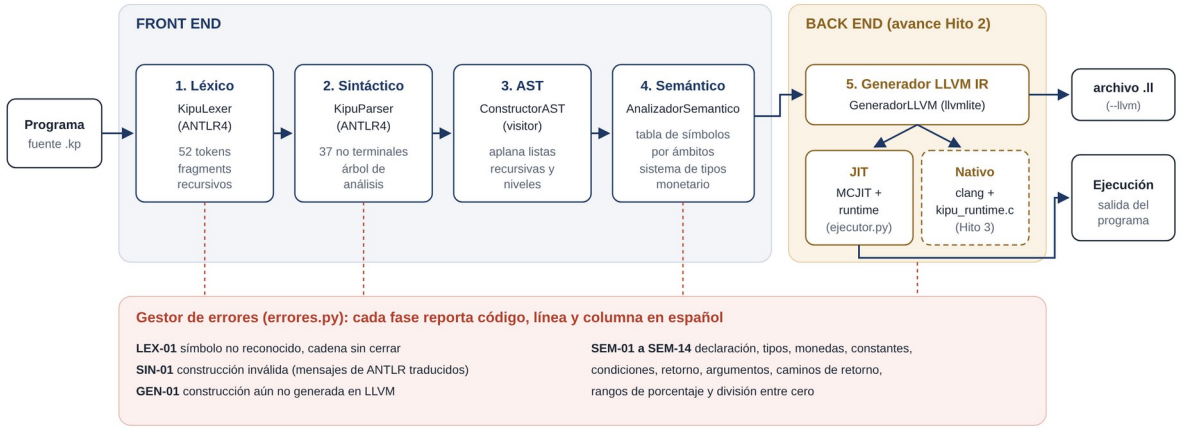


Figura 1. Arquitectura del compilador de Kipu

Tabla 9. Módulos del compilador

Módulo	Fase	Entrada	Salida
`grammar/Kipu.g4`	Especificación léxica y sintáctica	Gramática en forma estándar	KipuLexer y KipuParser generados por ANTLR 4.13.2
`src/generated/KipuLexer.py`	1. Análisis léxico	Texto del programa `.kp`	Flujo de tokens
`src/generated/KipuParser.py`	2. Análisis sintáctico	Flujo de tokens	Árbol de análisis sintáctico
`src/constructor_ast.py`, `src/ast_kipu.py`	3. Construcción del AST	Árbol de análisis	AST (aplana listas recursivas y niveles de precedencia)
`src/semantico.py`	4. Análisis semántico	AST	AST anotado con tipos y símbolos; tabla de símbolos

Módulo	Fase	Entrada	Salida
`src/generador_llvm.py`	5. Generación de código	AST anotado	Módulo LLVM IR (archivo `.ll`)
`src/ejecutor.py`, `runtime/kipu_runtime.c`	Ejecución	LLVM IR	Ejecución JIT (MCJIT) o ejecutable nativo con clang
`src/errores.py`	Transversal	Errores de cada fase	Mensajes en español: LEX-01, SIN-01, SEM-01 a SEM-14, GEN-01
`src/kipu.py`	Driver	Archivo y opciones	Orquesta las fases: `--tokens`, `--arbol`, `--simbolos`, `--llvm`, `--ejecutar`

Se introdujo un AST entre el parser y el analizador semántico por dos razones. Primero, la gramática en forma estándar produce árboles profundos (listas recursivas y un nivel por precedencia); el AST los reduce a nodos con significado, como `Binaria(op, izq, der)` o `Si(ramas, sino)`. Segundo, el análisis semántico y la generación de código trabajan sobre la misma estructura: el analizador anota el tipo de cada expresión y el símbolo de cada variable, y el generador usa esas anotaciones sin volver a calcularlas.

9. Plan de validación del compilador

Objetivo. Comprobar que cada fase del compilador acepta los programas correctos, rechaza los incorrectos con el error esperado y que el código generado produce los resultados correctos, con montos exactos al céntimo.

Estrategia. La validación se organiza por niveles, uno por fase, más la verificación estática de la gramática y la integración continua. Cada caso de prueba es un archivo `.kp` cuya primera línea declara el resultado esperado (`// espera: OK` o `// espera: SEM-03@4 SEM-03@5`, código@línea), de modo que la prueba compara el resultado exacto y no solo si hubo error. La Tabla 10 detalla el plan.

Tabla 10. Plan de validación por niveles

Nivel	Qué se valida	Casos y técnica	Criterio de aceptación
V1. Gramática	Lineamientos L1 a L6; ausencia de ambigüedad	`verificar_gramatica.py`; 35 ejemplos con detección exacta de ambigüedad	0 incumplimientos y 0 ambigüedades
V2. Léxico	Tokens y lexemas; errores LEX-01	7 archivos (palabras clave, literales, operadores, comentarios, inválidos) y 7 secuencias de tokens	Secuencia exacta de tokens; LEX-01 en la línea esperada
V3. Sintáctico	Aceptación y rechazo de construcciones	35 ejemplos de la Tabla 3, 3 programas válidos y 9 inválidos (falta ;, paréntesis, tipo inexistente, sino sin si, etc.)	Sin errores en válidos; primer SIN-01 en la línea esperada

Nivel	Qué se valida	Casos y técnica	Criterio de aceptación
V4. Derivaciones	Que la gramática genere cada ejemplo	28 derivaciones más a la izquierda verificadas paso a paso	Cada paso usa una producción de Kipu.g4
V5. Semántico	Los 14 errores semánticos y los programas correctos	7 programas válidos y 15 inválidos con 45 errores esperados; al menos 2 casos por código	Lista exacta código@línea; cobertura de los 14 códigos
V6. Generación de código	IR válido y resultados de ejecución	5 programas con salida esperada (aritmética, redondeo monetario, control de flujo, lógica, conversión), ejemplo de detracciones y construcción pendiente (GEN-01)	IR aceptado por el verificador de LLVM; salida idéntica a la esperada; JIT y ejecutable nativo coinciden
V7. Regresión	Que ningún cambio rompa lo anterior	GitHub Actions ejecuta V1 a V6 en Ubuntu y Windows en cada push	Todas las pruebas aprobadas

Casos de redondeo. Por ser el objetivo central del lenguaje, V6 incluye casos límite de redondeo half-up (alejándose de cero): `0.05 PEN * 50%` debe dar `0.03 PEN`, `-0.05 PEN * 50%` debe dar `-0.03 PEN`, `10.00 PEN / 3` debe dar `3.33 PEN` y `20.00 PEN / 3`, `6.67 PEN`.

Tabla 11. Matriz de trazabilidad (requisito del hito y evidencia)

Requisito	Evidencia en el repositorio	Estado actual
Gramática actualizada (forma estándar)	`grammar/Kipu.g4`, `scripts/verificar_gramatica.py`	Cumple L1 a L6
Derivaciones más a la izquierda (4 por gramática)	`scripts/derivaciones.py`, `docs/anexo_derivaciones.md`	28 de 28 verificadas
Analizador semántico con al menos 6 errores	`src/semantico.py`, `tests/semantico/`	14 errores; 45 de 45 detectados
Arquitectura del compilador	Sección 8, `src/kipu.py`	Implementada
Plan de validación	Sección 9, `tests/test_kipu.py`, `.github/workflows/pruebas.yml`	128 casos aprobados
Avance de la implementación (back end)	`src/generador_llvm.py`, `src/ejecutor.py`, `tests/llvm/`	Subconjunto generado y ejecutado

Tabla 12. Resultados actuales de la validación

Nivel	Casos	Resultado
V1. Gramática	1 verificación de lineamientos + 35 ejemplos sin ambigüedad	36 / 36
V2. Léxico	7 archivos + 7 secuencias de tokens	14 / 14
V3. Sintáctico	35 ejemplos + 3 válidos + 9 inválidos	47 / 47
V5. Semántico	7 válidos + 15 inválidos + cobertura de códigos	23 / 23

Nivel	Casos	Resultado
V6. Generación de código	5 programas con salida esperada + detracciones + GEN-01 + montos en céntimos	8 / 8
Total		128 / 128

Para el Hito 3 se ampliará V6 a las reglas, llamadas y `leer`, y se reportarán los resultados finales de la validación.

10. Avance de la implementación: generación de LLVM IR

Como avance del back end se implementó la generación de LLVM IR (Lattner y Adve, 2004) con la biblioteca llvmlite. El generador recorre el AST anotado y produce un módulo con una función `main`. La decisión central es la representación de los tipos de dominio (Tabla 13): los montos se guardan como enteros de 64 bits en céntimos, por lo que la suma y la resta son exactas, y la multiplicación por un porcentaje se redondea a céntimos con una función interna `kipu.div_redondeo` (redondeo half-up).

Tabla 13. Representación de los tipos de Kipu en LLVM IR

Tipo Kipu	Tipo LLVM	Representación
entero	i64	Entero con signo de 64 bits
decimal	double	Punto flotante de doble precisión
monto<M>	i64	Céntimos: `249.90 PEN` se guarda como 24990
porcentaje	i64	Fracción por 10 000: `18%` se guarda como 1800 y `12.5%` como 1250
booleano	i1	Verdadero o falso
texto	i8*	Cadena constante terminada en cero

El generador produce bloques básicos para `si`, `sino si`, `sino`, `mientras` y `para`, y evalúa `y` y `o` con cortocircuito mediante una instrucción `phi`. La salida de `mostrar` se delega a un runtime pequeño (`kipu_imprimir_*`), implementado en Python para la ejecución JIT y en C para el ejecutable nativo, con el mismo formato. El siguiente fragmento muestra el IR generado para calcular el IGV de un precio:

```
// Kipu
var precio: monto<PEN> = 249.90 PEN;
var igv: monto<PEN> = precio * 18%;
mostrar(precio + igv);

; LLVM IR generado (función main)
  store i64 24990, i64* %"precio"
  %"prod" = mul i64 %"precio.1", 1800
  %"r" = call i64 @"kipu.div_redondeo"(i64 %"prod", i64 10000)
  store i64 %"r", i64* %"igv"
  %"r.1" = add i64 %"precio.2", %"igv.1"
  call void @"kipu_imprimir_monto"(i64 %"r.1", i32 0)      ; imprime 294.88 PEN
```

La Figura 2 muestra la ejecución del ejemplo `ejemplos/detracciones.kp`, que liquida cuatro facturas con IGV, detracción del 12 % sobre las que superan S/ 700 y conversión del total a dólares. Los resultados se verificaron manualmente: por ejemplo, 3 × 249.90 = 749.70; el IGV de 134.946 se redondea a 134.95; el total de 884.65 supera el umbral y su detracción de 106.158 se redondea a 106.16.

```
$ python src/kipu.py ejemplos/detracciones.kp --ejecutar
Kipu 0.2.0 (Hito 2)  |  ejemplos/detracciones.kp
  [1/4] Análisis léxico      OK (178 tokens)
  [2/4] Análisis sintáctico  OK
  [3/4] Análisis semántico   OK
  [4/4] Generación LLVM IR   OK
LLVM IR escrito en ejemplos/detracciones.ll
--- Ejecución (JIT LLVM) ---
Factura 1 | total: 294.88 PEN | detracción: 0.00 PEN | neto: 294.88 PEN
Factura 2 | total: 589.76 PEN | detracción: 0.00 PEN | neto: 589.76 PEN
Factura 3 | total: 884.65 PEN | detracción: 106.16 PEN | neto: 778.49 PEN
Factura 4 | total: 1179.53 PEN | detracción: 141.54 PEN | neto: 1037.99 PEN
Total cobrado: 2701.12 PEN = 720.30 USD
--- Fin (código de salida 0) ---
Resultado: programa válido, sin errores.
```

Figura 2. Generación y ejecución JIT del ejemplo de detracciones

Tabla 14. Estado de la implementación al cierre del Hito 2

Componente	Estado	Detalle
Analizador léxico	Completo	52 tokens; fragments recursivos
Analizador sintáctico	Completo	37 no terminales, 92 producciones, sin ambigüedad
Construcción del AST	Completo	Todas las construcciones
Analizador semántico	Completo	14 errores, tabla de símbolos por ámbitos, análisis de caminos de retorno
Generación de LLVM IR	Parcial	Hecho: declaraciones, asignaciones, todas las expresiones, convertir, si, mientras, para, mostrar. Pendiente (Hito 3): reglas, retornar, llamadas, leer y concatenación de textos
Ejecución	Parcial	JIT con MCJIT operativo; ejecutable nativo con clang probado, se documentará en el Hito 3

11. Implementación, repositorio y uso

El repositorio (https://github.com/FabrizioBussalleu/kipu) contiene todos los artefactos de análisis, diseño y desarrollo: la gramática, el parser generado, el código fuente del compilador, los ejemplos, las pruebas, los scripts de verificación y la documentación de cada hito en `docs/`. El compilador está escrito en Python 3.10 o superior con `antlr4-python3-runtime 4.13.2` y `llvmlite`. Comandos principales:

```
python -m pip install -r requirements.txt
python src/kipu.py ejemplos/facturacion.kp --tokens --arbol --simbolos
python src/kipu.py tests/semantico/invalidos/06_errores_multiples.kp
```

```
python src/kipu.py ejemplos/detracciones.kp --ejecutar
python src/kipu.py ejemplos/detracciones.kp --llvm salida.ll
python -m unittest discover -s tests -v
python scripts/verificar_gramatica.py
python scripts/derivaciones.py
```

Gestión del proyecto. El código se gestiona en GitHub con un flujo de ramas: una rama `feature/...` por avance, integración en `develop` y entrega en `main` mediante pull requests. La integración continua (GitHub Actions) ejecuta la verificación de la gramática, las pruebas y las derivaciones en Ubuntu y Windows en cada cambio. El tablero de tareas por hito y responsable está en `docs/gestion_del_proyecto.md`.

La Figura 3 muestra el analizador semántico sobre un programa con cuatro errores de distinto tipo, reportados en una sola pasada.

```
 2  const UIT: monto<PEN> = 5350.00 PEN;
 3  var ventas: monto<PEN> = 0.00 PEN;
 4  var ventasUsd: monto<USD> = 0.00 USD;
 5  para mes desde 1 hasta 12 {
 6      UIT = 5500.00 PEN;
 7      ventas += ventasUsd;
 8      mostrar(totalAnual);
 9      var mes: entero = 1;
10  }

Kipu 0.2.0 (Hito 2)  |  tests/semantico/invalidos/06_errores_multiples.kp
  [1/3] Análisis léxico      OK (62 tokens)
  [2/3] Análisis sintáctico  OK
  [3/3] Análisis semántico   4 error(es)
[SEM-05] línea 6:5  No se puede modificar la constante 'UIT'.
[SEM-03] línea 7:15  Monedas incompatibles: no se puede sumar monto<PEN> con monto<USD>;
use convertir(...).
[SEM-01] línea 8:13  'totalAnual' no ha sido declarado.
[SEM-02] línea 9:9  'mes' ya fue declarado en este ámbito (línea 5).
Resultado: 4 error(es) encontrados.
```

Figura 3. Programa con errores semánticos y salida del compilador

12. Próximos pasos (Hito 3)

- Completar la generación de LLVM IR para reglas (funciones con parámetros y `retornar`), llamadas, `leer` y concatenación de textos.
- Generar ejecutables nativos con clang y el runtime en C, y documentar el proceso en Windows y Linux.
- Ejecutar el plan de validación completo y reportar sus resultados finales y las conclusiones.

Referencias

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). *Compilers: Principles, techniques, and tools* (2.ª ed.). Pearson.

Lattner, C. y Adve, V. (2004). LLVM: A compilation framework for lifelong program analysis and transformation. *Proceedings of the International Symposium on Code Generation and Optimization*, 75-86.

Parr, T. (2013). *The definitive ANTLR 4 reference*. The Pragmatic Bookshelf.

Superintendencia Nacional de Aduanas y de Administración Tributaria [SUNAT]. (s. f.). *Detracciones del IGV: Sistema de pago de obligaciones tributarias* [Cartilla]. https://orientacion.sunat.gob.pe/sites/default/files/inline-files/Cartilla_detracciones.pdf